

오픈소스 반도체 설계 및 제작 종합설계 3주차 학습노트

이수빈

1. 소프트웨어 언어와 하드웨어 언어

소프트웨어 언어는 CPU가 읽고 한 줄씩 순서대로 실행하는 명령이고, 하드웨어 언어는 만들어진 회로를 기술하는 말이라고 이해했다. 같은 for 문이라도 C에서는 시간에 따라 반복되지만 베릴로그에서는 회로 복사본으로 펼쳐져 동시에 동작한다. Q.10에서 외계인 25개를 판정하는 비교기가 25벌 생기는 것이 그 예였다.

하드웨어 언어로 쓴 알고리즘이 합성되려면 RTL이어야 한다. 레지스터에 값이 저장되고, 클럭 에지마다 조합 논리를 거쳐 다음 레지스터로 옮겨지는 구조로 쓰여 있어야 합성 도구가 게이트와 플립플롭으로 바꿀 수 있기 때문이다. ROM을 initial 블록으로 채운 코드가 시뮬레이션에서는 돌아가지만 합성할 때는 case 문으로 바꿔 써야 하는 것도 같은 이유다.

소프트웨어 언어로 하드웨어를 기술하려면 네 가지 개념이 필요하다.

a. 비트 단위의 객체 묘사. 하드웨어에서는 비트 폭이 곧 회로의 폭이다. 소프트웨어의 int는 알아서 넓어지지만 하드웨어는 설계자가 폭을 정해야 한다. 이번 실습에서 7'd128이 7비트로 잘려 0이 되는 바람에 세 번째 목숨 아이콘이 그려지지 않은 것이 그 예였다. SystemC에서는 sc_uint 같은 타입이 이 역할을 한다.

b. 다중치 논리. 하드웨어에는 0과 1 말고도 값이 정해지지 않은 상태(x)와 연결이 끊긴 상태(z)가 있다. 리셋 전 레지스터가 x인 것을 보면 초기화를 빠뜨린 곳을 찾을 수 있다. 다만 이번에 쓴 Verilator는 0과 1만 다루는 시뮬레이터라고 알고 있어서 x와 z를 직접 확인하지는 못했다.

c. 병렬성을 다루기 위한 시간 관리. 하드웨어는 모든 블록이 동시에 동작하기 때문에 언제 동작하는지를 정해 주는 클럭과 시뮬레이션 시간이 필요하다. 베릴로그에서는 클럭 에지와 논블로킹 할당이 같은 순간에 값을 함께 갱신해 준다. Q.11의 문제도 결국 시간 관리에서 나왔다. RTL은 20ns마다 움직이는데 키보드는 약 16.6ms마다 읽혔다.

d. 상태와 더불어 사건의 관리. 값뿐 아니라 값이 바뀌는 순간도 다뤄야 한다. 스페이스를 꺾 눌러도 총알이 한 발만 나가게 한 것은 버튼의 상승 에지를 잡는 엣지 검출이고, 게임의 진행은 FSM의 상태 전이로 관리했다. SystemC에서는 sensitive와 wait가 이런 사건에 반응하는 방법이다.

2) Q.1과 Q.4의 베릴로그 RTL이 그래픽 장치를 다루는 방식의 차이

가장 큰 차이는 그래픽 평면의 좌표가 입력이냐 출력이냐이다.

Q.1은 VGA가 지금 그리는 좌표를 RTL에 입력으로 주고, RTL은 그 좌표가 캐논인지 외계인인지 총알인지를 조합 회로로 비교해서 색을 바로 돌려준다. 프레임을 저장하지 않고 좌표가 바뀔 때마다 색을 새로 계산하며, 좌표의 타이밍은 RTL이 아니라 외부가 정한다. 해상도는 640x480이어서 한 프레임에 42만 클럭 정도가 걸린다. Q.4는 RTL이 스캔 카운터를 두고 128x64 화면 전체를 스스로 훑으면서, 각 점의 좌표와 값을 출력으로 내보낸다. 좌표의 주도권이 외부에서 RTL로 넘어온 것이다. 한 프레임은 8,192 클럭이다.

이 차이를 내부와 외부의 관점으로 보면 더 분명해진다. 내부는 클럭과 신호, 병렬로 동작하는 프로세스가 있는 하드웨어의 세계이고, 외부는 SDL 창이나 키보드 같은 일반 소프트웨어의 세계다. 게임 로직과 스프라이트 판정은 두 경우 모두 내부에 있다. 다른 점은 화면 쪽 일을 누가 맡느냐였다. Q.1에서는 RTL이 바깥에서 오는 좌표에 응답만 했고, Q.4에서는 RTL이 화면 순회를 직접 하고 받은 데이터를 저장하고 표시하는 일은 바깥이 맡았다. 이 구조 때문에 Q.4에서는 RTL의 속도와 외부 키보드의 속도 차이가 문제로 드러났다고 생각한다.

정리하면 Q.1은 좌표를 받아 색을 계산해 돌려주는 조합 회로 중심의 구조이고, Q.4는 카운터와 FSM으로 화면을 직접 훑으며 쓰는 순차 회로 중심의 구조다. 좌표의 입출력 방향이 다르다는 것은 그래픽을 다루는 방식 자체가 다르다는 뜻이다.

3. SystemC에 대한 이해

SystemC는 C++로 하드웨어를 묘사하기 위해 위의 네 가지 개념을 갖춘 클래스 라이브러리이고, 이것을 시뮬레이션하는 병렬 시뮬레이션 엔진도 함께 들어 있다고 이해했다. 비트 객체, 시간, 병렬 프로세스와 사건이 모두 라이브러리로 제공된다.

이번 실습에서 느낀 장점은 세 가지다. 첫째, 회로로 만들 RTL이 키보드와 화면이 있는 컴퓨터 환경 안에서 곧바로 동작한다. Verilator가 RTL을 C++ 모델로 바꿔 주면 SystemC 테스트벤치가 GLCD 화면과 키보드를 거기에 붙였다. 둘째, 실제 보드가 없어도 게임을 화면에 띄워 보면서 검증할 수 있다. 셋째, 테스트벤치를 C++로 쓸 수 있어서 SDL 같은 기존 라이브러리를 그대로 쓸 수 있다.

4. Q.10 스프라이트 시현 방식: 아두이노 C++와 베릴로그 RTL

아두이노에서는 스프라이트를 비트맵 배열로 저장해 두고, drawBitmap 같은 함수를 호출해서 그린다. CPU가 명령을 순서대로 실행하면서 8x8 픽셀을 하나씩 화면 버퍼에 쓰고, 그린 결과는 버퍼에 남는다. 캐논을 그린 뒤 외계인을 그리는 식으로 호출 순서가 곧 겹침의 우선순위가 된다. 베릴로그 RTL에서는 스프라이트 비트맵을 ROM에 두고, 스캔 카운터가 가리키는 현재 좌표가 스프라이트 영역 안인지를 매 클럭 비교기로 판정한다. 영역 안이면 ROM에서 그 위치의 비트를 읽어 점의 값을 정한다. 그림을 저장하지 않고, 스캔이 그 좌표를 지나가는 순간에 값을 계산해서 내보내는 방식이다.

객체가 여러 개일 때의 차이도 컸다. 아두이노에서 외계인 25개는 같은 함수를 25번 호출하는 것이라 자원은 그대로이고 시간이 더 걸린다. RTL에서는 같은 일이 비교기 25벌로 펼쳐져 매 클럭 동시에 동작한다. 소프트웨어는 시간을 써서 일을 나누고, 하드웨어는 면적을 써서 일을 나눈다는 점이 가장 큰 차이라고 생각한다.

이 차이를 보면서 그림을 저장하지 않는데 화면에는 어떻게 그림이 남아 있는지가 궁금했다. 코드를 따라가 보니 RTL이 화면 전체를 훑는 8,192 클럭이 쉬지 않고 반복되면서 같은 그림을 매 프레임 새로 내보내고 있었고, 그 값을 받아 저장하는 곳은 RTL 바깥의 GLCD 모델이었다. 아두이노는 한 번 그려 두고 유지하는 방식이고, RTL은 매 프레임 처음부터 다시 그리는 방식인 셈이다. 또 외계인 25개가 비교기 25벌로 펼쳐진다면 실제 칩에서 면적이 얼마나 될지도 궁금하다.

5. Q.11 좌우 화살표 키보드 입력이 이상하게 반응하는 이유

Q.11을 실행하고 좌우 방향키를 짧게 눌러도 캐논이 바로 한쪽 벽에 붙어 버렸고, 가운데에서 멈출 수가 없었다.

원인은 두 쪽의 시간 척도가 달랐기 때문이다. RTL의 캐논 이동 코드에는 속도를 제한하는 장치가 없어서 버튼이 눌러 있는 동안 매 클럭 1픽셀씩 움직였다. 50MHz 기준으로 화면 폭에 가까운 거리를 가는 데 몇 마이크로초면 충분하다. 반면 키보드는 외부에서 약 16.6ms마다, 클럭으로 치면 80만 번 넘게 지난 뒤에야 한 번 읽혔다. 사람이 키를 눌렀다 때는 짧은 시간에도 RTL은 수십만 클럭을 지나기 때문에, 한 번의 입력이 수만 번의 이동으로 불어나서 캐논이 끝까지 가 버린 것이다.

소프트웨어에서는 루프 한 번에 입력을 한 번 읽고 한 칸 움직이면 되니까 이런 일이 생기

지 않는다. 하드웨어는 클럭마다 모든 조건이 동시에 평가되기 때문에, 동작 속도를 일부러 제한하거나 입력을 프레임 단위로 맞춰 주지 않으면 이동 속도가 클럭 주파수에 그대로 묶인다고 이해했다. 이 원인을 찾으면서 한 가지 의문이 들었다. 이것은 RTL의 문제일까, 테스트벤치의 문제일까 하는 것이다. 처음에는 RTL이 잘못 만들어졌다고 생각했다. 그런데 클럭마다 한 칸씩 움직이는 코드는 문법상으로도 논리상으로도 틀린 곳이 없었고, 테스트벤치가 키보드를 16.6ms마다 읽는 것도 키보드 입력을 다루는 방식으로는 평범했다. 둘 중 하나만 보면 문제가 없는데 합쳐 놓으면 문제가 생긴 것이다.

이것을 확인하려고 Q.13의 RTL 세 버전을 직접 돌려 봤다. 원본인 버전 0에서는 캐논이 벽에 붙었고, 프레임이 끝날 때 버튼을 한 번 읽도록 바꾼 버전 1에서는 캐논이 부드러워졌지만 이번에는 총알이 보이지 않았다. 총알이 여전히 매 클럭 움직여서 한 프레임 안에 화면 위로 사라졌기 때문이라고 생각한다. 총알 속도까지 제한한 버전 4에서야 둘 다 정상으로 동작했다. 같은 문제가 캐논에서 해결되자 총알에서 다시 나타난 것을 보면 시간 척도가 맞지 않는 곳은 한 군데가 아니었다. 테스트벤치도 키보드 입력이 이벤트가 생기는 즉시 반영되도록 바뀌어 있었으니, 결국 한쪽만 고친 것이 아니라 양쪽을 함께 맞춘 것이었다.

남은 의문은 이런 문제를 누가 찾아야 하는가이다. RTL만 보는 사람도, 테스트벤치만 보는 사람도 각자는 맞다고 할 것이다. 둘이 각각은 맞고 합쳤을 때만 틀리면 어느 한쪽만 확인해서는 찾을 수 없다. 또 이것은 시뮬레이션에서 확인한 것이어서 실제 보드에서 버튼과 클럭의 관계가 어떻게 다를지는 아직 확인하지 못했다. 앞으로는 문제가 생겼을 때 RTL이나 테스트벤치 중 한쪽만 의심하지 않고, 둘이 서로 어떤 속도로 맞물리는지도 함께 확인해 보려고 한다.

6. Q.13 시뮬레이션: 테스트벤치와 RTL의 변경점

먼저 Q.11과 비교해 SystemC 테스트벤치에서 달라진 점이다. 키보드 입력이 GLCD 모델 안으로 들어가서, 키가 눌리거나 떴어지는 이벤트가 발생하는 즉시 버튼 신호가 바뀌도록 되어 있었다. 키 상태가 프레임 주기에 묶이지 않게 된 것이다. 화면도 클럭마다 점을 찍다가 프레임의 마지막 점에서 갱신하도록 되어 있어서 RTL의 스캔 동작과 화면 갱신이 맞아 떨어졌다. 클럭은 10MHz이고, 최상위 테스트벤치는 리셋과 프레임 카운터만 남은 단순한 형태였다.

다음은 키보드 입력을 보완하기 위한 RTL의 차이다. Q.13에는 RTL이 다섯 버전으로 들어 있었고, 버전마다 문제를 하나씩 고치고 있었다. 버전 0은 캐논과 총알이 매 클럭 움직이

는 원본이다. 버전 1은 프레임이 끝날 때 버튼을 한 번 읽는 상태를 추가해서, 캐논이 프레임당 한 번만 움직이게 했다. 버전 2는 스페이스를 누른 순간만 잡는 엣지 검출을 넣었다. 버전 3은 하나의 상태 신호를 여러 곳에서 바꾸던 것을 한 곳에서만 바꾸도록 정리해서 FPGA 합성에서 문제가 되는 다중 구동을 없앴다. 버전 4는 총알의 이동 속도를 `game_tick`으로 제한해서 시간 척도를 맞췄다.

직접 돌려 본 결과는 이렇다. 버전 0에서는 캐논이 벽에 붙고 총알이 빠르게 보였다. 버전 1에서는 캐논이 부드럽게 움직였지만 총알이 보이지 않았다. 총알이 여전히 매 클럭 움직여서 한 프레임 안에 화면 위로 사라졌기 때문이라고 생각한다. 버전 4에서는 캐논이 부드럽고 총알도 적당한 속도로 올라갔으며, 스페이스를 꼭 눌러도 한 발씩만 나갔다.

결국 이번 실습의 핵심은 외부의 키보드와 내부의 RTL 사이의 시간 척도 차이를 테스트벤치와 RTL 양쪽에서 맞춰 준 것이라고 생각한다. 시뮬레이션에서 게임처럼 보이는 것과 그 속도 관계를 이해하고 있는 것은 다른 일이라는 것을 이번에도 확인했다.